

JavaScript

Chapitre 03

de ES5 à ES6

De ECMAScript 5 à ECMAScript 6 (ECMAScript 2015)

Ce chapitre se focalise sur le passage de ES5 à ES6, en particulier les nouveautés apportées par la nouvelle norme.

- ES6 maintient la compatibilité avec ce qui précède (ES1, 2, 3, 5)
- ES6 permet une adoption progressive des nouveaux éléments

<http://kangax.github.io/compat-table/es6/>

- C'est une évolution majeure du langage → ES6 va modifier de manière significative la façon d'écrire du JS

Seul les éléments les plus importants vis à vis du cours sont présent ici. Pour une liste complète, référez-vous à la spécification : **<http://www.ecma-international.org/ecma-262/6.0/>**

Linkographie

Ce chapitre s'appuie sur les références suivantes

- **<https://hacks.mozilla.org/2015/04/es6-in-depth-an-introduction/>**
traduit en français ici : **<https://tech.mozfr.org/post/2015/05/19/ES6-en-details-%3A-Une-introduction>**
- **<https://github.com/lukehoban/es6features>**
- **<http://es6-features.org>**
- **<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>**

Constantes

ES6 introduit dans la norme la notion de constante.

```
// ECMAScript 5
//  only in ES5 through the help of object
properties
//  and only in global context and not in a block
scope
Object.defineProperty(typeof global === "object" ?
global : window, "PI", {
  value:      3.141593,
  enumerable: true,
  writable:   false,
  configurable: false
})
PI > 3.0; //true

// ECMAScript 6
const PI = 3.141593;
PI > 3.0; //true
```

Portée des variables

ES6 introduit l'instruction **let** qui permet de définir des variables dont la portée est le bloc d'instruction contrairement à **var** qui produit des variables dont la portée est la fonction.

```
// ECMAScript 5
var i, x;
for (i = 0; i <
a.length; i++) {
  x = a[i];
  ...
}

var callbacks = [];
for (var i = 0; i <= 2;
i++) {
  (function (i) {
    callbacks[i] =
function() { return i *
2; };
  })(i);
}
callbacks[2]() === 4;
```

```
// ECMAScript 6
for (let i = 0; i <
a.length; i++) {
  let x = a[i];
  ...
}

let callbacks = [];
for (let i = 0; i <= 2;
i++) {
  callbacks[i] =
function () { return i
* 2; };
}
callbacks[2]() === 4;
```

Définition d'une variable : **let** vs **var**.

Portée des fonctions

Avec ES6, les fonctions définies dans un bloc d'instruction ont maintenant comme portée ce bloc. Cela permet de lever l'ambiguïté du cas ci-dessous.

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions>

```
//ECMAScript 6
{
  function foo () {
    return 1; }
  foo() === 1; //true
  {
    function foo () {
      return 2; }
    foo() === 2; //
true
  }
  foo() === 1; //true
}
```

```
//Unspecified behavior
if (shouldDefineZero) {
  function zero() {
    // DANGER:
    compatibility risk
    console.log("This
is zero.");
  }
}

// Safe way to define
zero
var zero;
if (shouldDefineZero) {
  zero = function() {
    console.log("This
is zero.");
  };
}
```

ES6 et la portée des fonctions.

Itérateur et boucle `for...of`

Contrairement à la boucle `for...in` qui boucle sur les propriétés énumérable d'un objet, la boucle `for...of` boucle sur les données contenues dans l'objet itérable considéré.

```
// Iterating over an
Array
let iterable = [10, 20,
30];

for (let value of
iterable) {
  value += 1;
  console.log(value);
}
// 11
// 21
// 31
```

```
// Iterating over a
String
let iterable = 'boo';

for (let value of
iterable) {
  console.log(value);
}
// "b"
// "o"
// "o"
```

```
// Iterating over a DOM collection (NodeList)
// Note: This will only work in platforms that have
// implemented NodeList.prototype[Symbol.iterator]
let articleParagraphs =
document.querySelectorAll('article > p');

for (let paragraph of articleParagraphs) {
  paragraph.classList.add('read');
}
```

Utilisation du `for...of`.

Itérateur personnalisé

Il est possible dorénavant de définir son propre itérateur.

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...of>

Exemple

```
//Iterating over other iterable objects
var iterable = {
  [Symbol.iterator]() {
    return {
      i: 0,
      next() {
        if (this.i < 3) {
          return { value: this.i++, done: false };
        }
        return { value: undefined, done: true };
      }
    };
  }
};
```

```
for (const value of iterable) {
  console.log(value);
}
// 0
// 1
// 2
```

Fonction fléchée (arrow function)

ES 6 introduit la fonction fléchée symbolisé par `=>`.

Attention, il y ne s'agit pas uniquement d'un nouveau symbole pour définir une fonction. Cette dernière possède des propriétés différentes, en particulier vis-à-vis du `this`.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

```
//ECMAScript 5
nums.forEach(function (v) {
  if (v % 5 === 0)
    fives.push(v);
});
```

```
//ECMAScript 6
nums.forEach(v => {
  if (v % 5 === 0)
    fives.push(v);
})
```

Les littéraux

ES2015 apporte plusieurs améliorations qui impactent les littéraux (définition du prototype à la construction, définition courte, définition de méthodes, propriétés calculées, etc.).

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types#Object_literals

```
// Enhanced Object
literals
var obj = {
  // __proto__
  __proto__:
theProtoObj,
  // Shorthand for
  'handler: handler'
  handler,
  // Methods
  toString() {
    // Super calls
    return 'd ' +
super.toString();
  },
  // Computed (dynamic)
  property names
  [ 'prop_' + (() =>
42)() ]: 42
};
```

```
// ECMAScript 5
parseInt("111110111",
2) === 503; //true
parseInt("767", 8) ===
503; //true
0767 === 503; // true :
only in non-strict
```

```
// ECMAScript 6
0b111110111 === 503;
//true
0o767 === 503; //true
```

ES6 et les littéraux.

Gabarit de chaîne de caractères

Avec ES6, les gabarits de chaînes de caractères apparaissent !

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals

Définition

```
// Basic literal string creation
console.log(`In JavaScript '\n' is a line-feed.`);
//In JavaScript '
//' is a line-feed.

// Multiline strings
console.log(`In JavaScript this is
not legal.`);
//In JavaScript this is
// not legal.

// String interpolation
var name = "Bob", time = "today";
console.log(`Hello ${name}, how are you ${time}?`);
//Hello Bob, how are you today?
```

Personnalisation

Il est possible d'analyser le gabarit avec une fonction personnalisée.

```
var person = 'Mike';
var age = 28;

function myTag(strings, personExp, ageExp) {
  var str0 = strings[0]; // "that "
  var str1 = strings[1]; // " is a "

  var ageStr;
  if (ageExp > 99){
    ageStr = 'centenarian';
  } else {
    ageStr = 'youngster';
  }

  return str0 + personExp + str1 + ageStr;
}

console.log(myTag`that ${ person } is a ${ age }`);
//that Mike is a youngster
```

Paramètres par défauts

Avec ES6 arrive la notion de paramètres par défaut dans la définition des fonctions !

```
function multiplier(a, b = 1) {  
    return a * b;  
}
```

```
multiplier(5, 2); // 10  
multiplier(5, 1); // 5  
multiplier(5); // 5
```

```
//Parameters already encountered are available to  
later default parameters
```

```
function singulierAutoPluriel(singulier, pluriel =  
singulier+"s",  
                                rallyingCry = pluriel +  
" B00U :) !!!") {  
    return [singulier, pluriel, rallyingCry ];  
}
```

```
var t = singulierAutoPluriel("Gecko");  
console.log(t); //["Gecko", "Geckos", "Geckos B00U  
:) !!!"]
```

Nombre variable d'arguments

Il est désormais possible de spécifier qu'une fonction prend un nombre variable d'arguments. On peut ensuite les traiter avec la boucle **for...of** par exemple.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/rest_parameters

```
//Rest
function sum(...theArgs) {
  return theArgs.reduce((previous, current) => {
    return previous + current;
  });
}

console.log(sum(1, 2, 3)); // expected output: 6
console.log(sum(1, 2, 3, 4)); // expected output:
10
```

Affectation par décomposition

L'affectation par décomposition permet d'extraire directement des données d'un tableau ou d'un objet en spécifiant les associations.

dernière mise à jour le 02/09/2024

DE ES5 à ES6 / B. Desbenoit

```
//Array matching
var a, b, rest;
[a, b] = [10, 20];
console.log(a); //
expected output: 10
console.log(b); //
expected output: 20

[a, b, ...rest] = [10,
20, 30, 40, 50];
console.log(rest); //
expected output:
[30,40,50]

[a, b] = [b, a];
console.log(a); //
expected output: 20
console.log(b); //
expected output: 10
```

```
//Objet matching
var o = {p: 42, q:
true};
var {p, q} = o;
console.log(p); // 42
console.log(q); // true

// Assign new variable
names
var {p: toto, q: truc}
= o;
console.log(toto); //
42
console.log(truc); //
true
```

ES6 et l'affectation par décomposition.

Affectation par décomposition et paramètres par défaut

On peut combiner l'affectation par décomposition et les paramètres par défaut dans la définition d'une fonction.

```
//Default parameters
function dessinGrapheES2015({size = 'big', cords =
{ x: 0, y: 0 }, radius = 25} = {}) {
  console.log(size, cords, radius);
}

dessinGrapheES2015({
  cords: { x: 18, y: 30 },
  radius: 30
});
```

Map, Set, WeakMap et WeakSet

ES6 introduit plusieurs structures de données utiles pour implanter des algorithmes classiques : **Set**, **Map**, **WeakSet**, **WeakMap**.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Keyed_collections

```
// Set
let s = new Set();
s.add("hello").add("goodbye").add("hello");
s.size === 2; //true
s.has("hello") === true; //true
for (let key of s.values()) //
insertion order
    console.log(key);
//hello
//goodbye
```

```
// Map
var myMap = new Map();
myMap.set(0, 'zero');
myMap.set(1, 'one');

for (var [key, value] of myMap) {
    console.log(key + ' = ' + value);
}
// 0 = zero
// 1 = one

myMap.forEach(function(
value, key) {
    console.log(key + ' = ' + value);
});
// 0 = zero
// 1 = one
```

Utilisation de **Set** et **Map**.

WeakMap, **WeakSet** - Contrairement à **Set** et **Map**, les valeurs(respectivement les clés) de l'ensemble sont des objets et sont référencé de manière faible. De fait

l'objet est nettoyé des lors que seul la collection y fait référence. De plus, il n'est pas possible de lister les éléments de l'ensemble.

ES6 et les classes

Bien qu'il n'y ai pas à proprement parler de nouvelles possibilités ici, ES6 apporte de nombreux changement syntaxiques (bienvenus !) en ce qui concerne les classes.

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>

```
class SkinnedMesh extends THREE.Mesh {
  constructor(geometry, materials) {
    super(geometry, materials);

    this.idMatrix = SkinnedMesh.defaultMatrix();
    this.bones = [];
    this.boneMatrices = [];
    //...
  }
  update(camera) {
    //...
    super.update();
  }
  get boneCount() {
    return this.bones.length;
  }
  set matrixType(matrixType) {
    this.idMatrix = SkinnedMesh[matrixType]();
  }
  static defaultMatrix() {
    return new THREE.Matrix4();
  }
}
```

Speaker notes